Ejercicio 1 Simple Factory

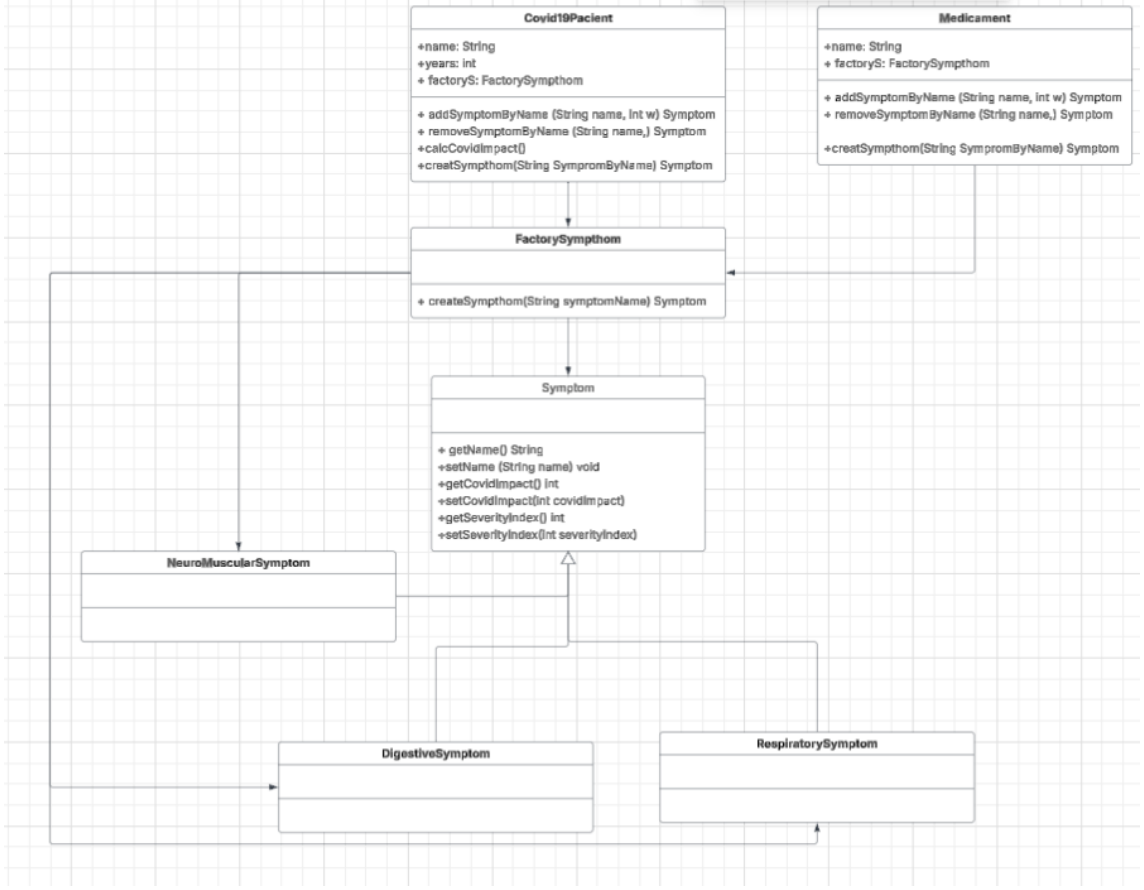
En el diseño original se incumple el principio de responsabilidad única ya que la clase Covid19Pacient se encarga tanto de gestionar los síntomas como de crearlos. Además, si se quiere añadir un nuevo tipo de síntoma (respiration) nos obliga a modificar el método createSymptom() de la clase covid19Pacient, mostrando que no esta cerrado ante modificaciones.

Para solucionar esto hay que aplicar un patrón Simple Factory en el que la nueva clase Factory solo se encargara de crear un síntoma y devolverlo. De esta manera, el resto de las clases (medicament y Covid19Pacient) delegan en la fábrica el proceso de crear el objeto, y además no necesitan conocer las subclases concretas. Y si se crea un nuevo síntoma solo hay que cambiar la clase Factory, y si es un nuevo tipo se crea e implementa la clase sintom, pero ya no hace falta cambiar código en covid19pacient ni en medicament.

Para implementar mareos sirve con actualizar en la clase FactorySympthom añadiendolo

```
List<String> impact1 = Arrays.asList("nauseas", "vómitos", "congestión nasal","diarrea","hemoptisis","congestion conjuntival","mareos");
List<Double> index1 = Arrays.asList(5.0, 4.8, 3.7, 0.9, 0.8,0.8,0.8);

List<String> digestiveSymptom=Arrays.asList("nauseas", "vómitos","diarrea");
List<String> neuroMuscularSymptom=Arrays.asList("fiebre", "astenia", "cefalea", "mialgia","escalofrios","mareos");
```



Para el apartado c, que use el mismo síntoma ya creado tanto en medicament como en Covid19Pacient lo que he hecho es añadir un map<String,Symptom> donde se guardan los síntomas ubicados por el nombre, entonces en cada vez que se crea trata de buscarlo, si existe lo devuelve, sino creamos el nuevo síntoma y lo añadimos.

```
private static Map<String, Symptom> symptomsByName = new HashMap<>();


public Symptom createSymptom(String symptomName) {

    Symptom sintomaEncontrado=symptomsByName.get(symptomName);

    if(sintomaEncontrado==null) {

    List<String> impact5 = Arrays.asList("fiebre", "tos seca", "astenia","expectoracion");
    List<Double> index5 = Arrays.asList(87.9, 67.7, 38.1, 33.4);
    List<String> impact3 = Arrays.asList("disnea", "dolor de garganta", "cefalea","mialgia","escalofrios");
    List<Double> index3 = Arrays.asList(18.6, 13.9, 13.6, 14.8, 11.4);
    List<String> impact1 = Arrays.asList("nauseas", "vómitos", "congestión nasal","diarrea","hemoptisis","congestion conjuntival","mareos");
    List<Double> index1 = Arrays.asList(5.0, 4.8, 3.7, 0.9, 0.8,0.8,0.8);

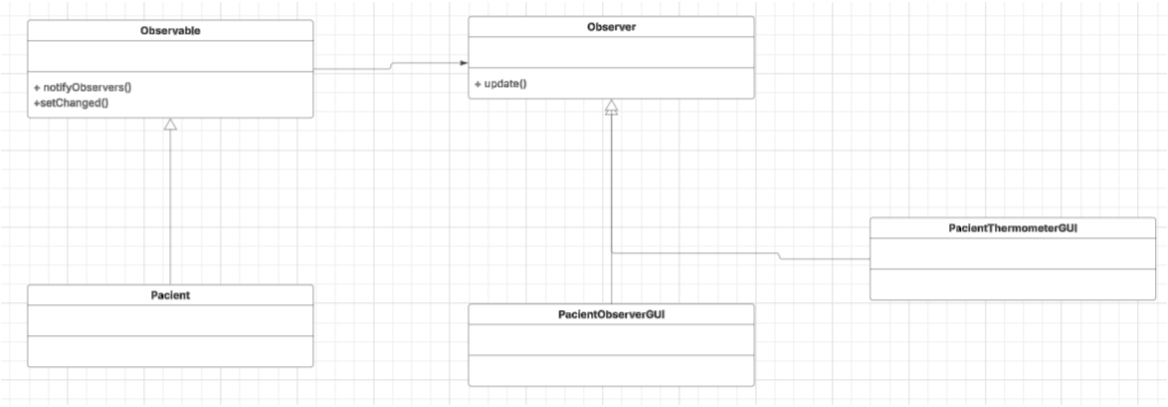
    List<String> digestiveSymptom=Arrays.asList("nauseas", "vómitos","diarrea");
    List<String> neuroMuscularSymptom=Arrays.asList("fiebre", "astenia", "cefalea", "mialgia","escalofrios","mareos");
    List<String> respiratorySymptom=Arrays.asList("tos seca","expectoracion","disnea","dolor de garganta", "congestión nasal","hemoptisis","congestion conj

    int impact=0;
    double index=0;
    if (impact5.contains(symptomName)) {impact=5; index= index5.get(impact5.indexOf(symptomName));}
      else if (impact3.contains(symptomName)) {impact=3;index= index3.get(impact3.indexOf(symptomName));}
        else if (impact1.contains(symptomName)) {impact=1; index= index1.get(impact1.indexOf(symptomName));}


    if (impact!=0)  {
        if (digestiveSymptom.contains(symptomName)) { sintomaEncontrado= new DigestiveSymptom(symptomName,(int)index, impact);}
        if (neuroMuscularSymptom.contains(symptomName)) { sintomaEncontrado= new NeuroMuscularSymptom(symptomName,(int)index, impact);}
        if (respiratorySymptom.contains(symptomName)) { sintomaEncontrado= new RespiratorySymptom(symptomName,(int)index, impact);}
        symptomsByName.put(symptomName,sintomaEncontrado);
        return sintomaEncontrado;
    }
    return null;
    }
    else {
        return sintomaEncontrado;
    }
}
```

Ejercicio 2 Observer

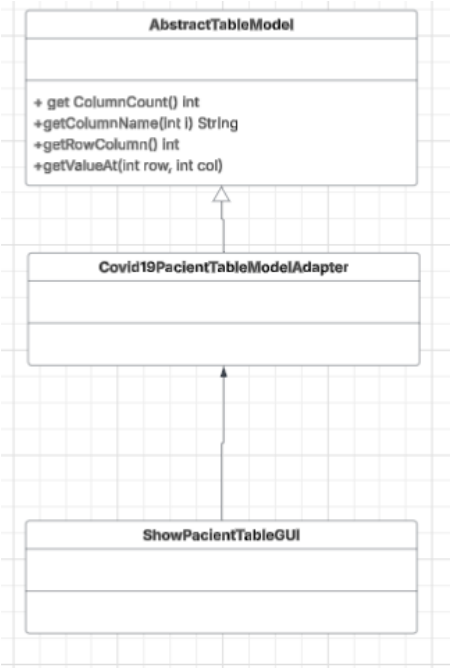
Se ha implementado el patrón observer, para que las interfaces graficas se actualicen en caso de que el paciente cambie.



Ejercicio 3 Adapter

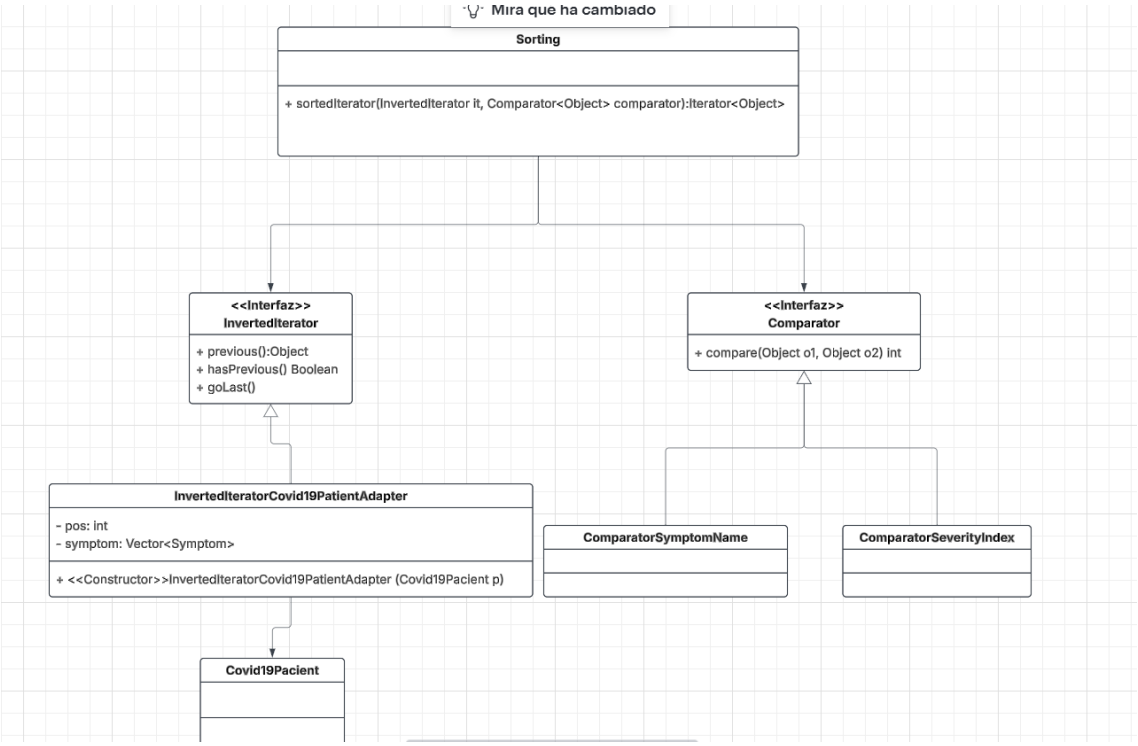
Se ha creado Covid19PacientTAbleModelAdapter para adaptar los datos del paciente al formato que necesita Jtable. El adaptador recibe un paciente y sobrescribe los métodos de AbstatTableModel para indicar la información.

Y para actualizar la tabla automáticamente sirve con añadirlo al patrón observer de antes, es decir que implemente la interfaz y añadirle el método update() para que vuelva a poner los datos en la tabla



Ejercicio 4 Patrón Iterator y Adapter

Para poder aplicar sorted a los síntomas del paciente se debe hacer que implemente la clase InvertedIterator, que para ello se aplica el patrón adapter para poder aplicarlo. Además de que existan Comparator que usen a los síntomas, en este caso implementan la interfaz correspondiente.


Mira que ha cambiado
Sorting
+ sortedIterator(InvertedIterator it, Comparator<Object> comparator):Iterator<Object>
<<Interfaz>>
InvertedIterator
+ previous():Object
+ hasPrevious() Boolean
+ goLast()
<<Interfaz>>
Comparator
+ compare(Object o1, Object o2) int
InvertedIteratorCovid19PatientAdapter
- pos: int
- symptom: Vector<Symptom>
+ <<Constructor>>InvertedIteratorCovid19PatientAdapter (Covid19Pacient p)
ComparatorSymptomName
ComparatorSeverityIndex
Covid19Pacient